Command line. Part I

Bioinformatics course
Term 2, 2023
Skoltech

Marina Pak
PhD-4

Seminar credits:

Lisa Grigorashvili
Dasha Nikolaeva
Asia Mendelevich

Outline

Intro

- What is a command line
- Why ~~you have no choice but use a command line~~ command line is useful
- What is a computational cluster

Command line how-to

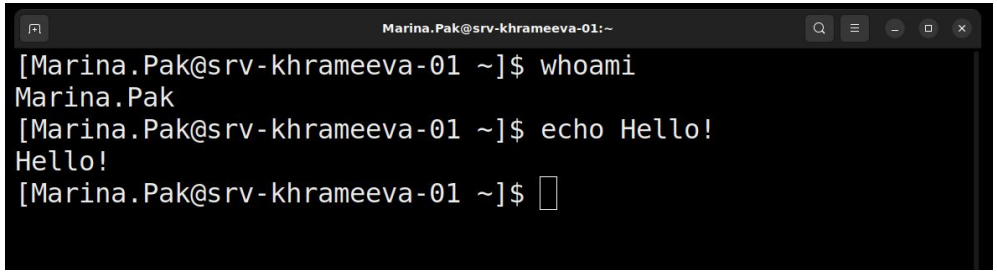
Nov, 10:

- Directories
- Operations with files and directories
- Text processing

Dec, 1:

- Scripting
- GUI for working on a cluster
- Some other useful commands and tips

What is a command line



Today we will type **specific words** into a **black window** to obtain a **desired result**.

Commands

Directives to a computer to perform a specific task.

Command-line interface (CLI or shell or terminal)

Processes commands to a computer program in the form of lines of text.

CLI is opposed to graphical user interface (GUI).

CLI's are different in Windows and Unix systems (Linux, MacOS).

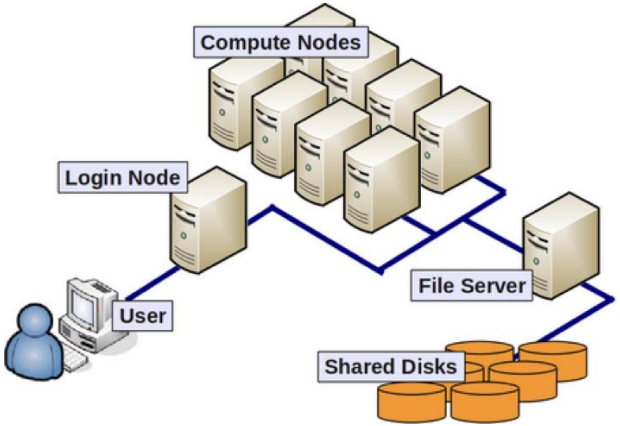
Commands are interpreted by a computer program - **command-line interpreter**. Most of the modern Unix-like systems use **Bash** interpreter.

Why command line

- Using cmd is inevitable: most of bioinformatics tools are designed for command line
- Automatization and big data
    - GUI cannot handle thousands of files
    - Biological data is big
- Computational power

Computational cluster (HPC cluster / server)

Zhores HPC - #9 position in computation capacity in Russia (1 PFLOPS). hpc.skoltech.ru



https://hbctraining.github.io/Intro-to-shell-flipped/lessons/08_HPC_intro_and_terms.html

A computer cluster is a set of computers (nodes) that work together so that they can be viewed as a single system.

Good things about clusters:

- Computational power
- Large file space
- Remote connection

Command line how-to

- Google, Stackoverflow, RTFM, ChatGPT.
- Commands are easy to guess and memorize (e.g. *cd* = change directory, *rm* = remove, *history* = history of commands).
- Retype commands, not copy-paste (otherwise you won't memorize them).

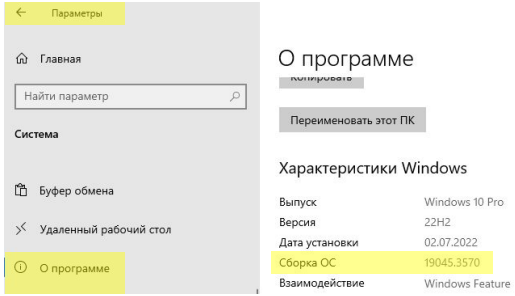
Safety:

- Don't use commands in blind! Know what they do.
- You are not alone at the server. Don't run computationally demanding commands. Mind big files.
- Do backups.

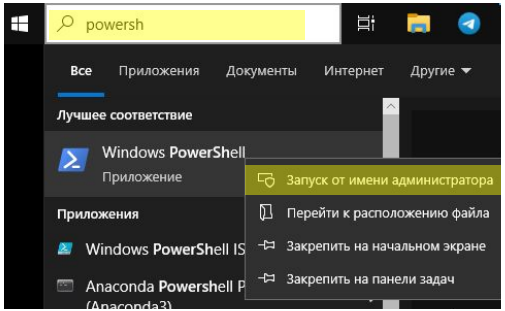
Log in to cluster

Installing Windows Subsystem for Linux (WSL)

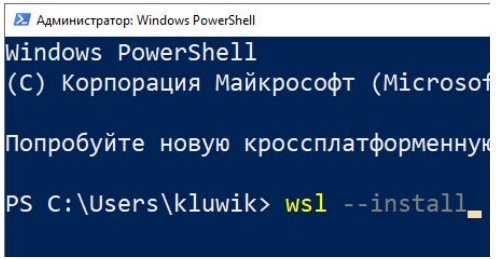
1. Make sure that you are running **Windows 11** or **Windows 10 Build 19041 and higher**. To check the build go to Start>Settings>System>About.
For older versions use this guide:
https://learn.microsoft.com/en-us/windows/wsl/install-manual
https://learn.microsoft.com/ru-ru/windows/wsl/install-manual



2. Run Windows **PowerShell** as an **administrator** (right-click > Run as administrator.

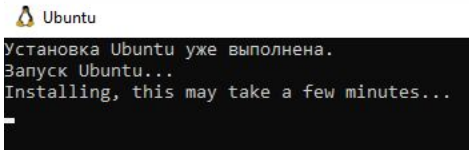


3. Type **`wsl --install`** and press Enter. Installation takes several minutes.

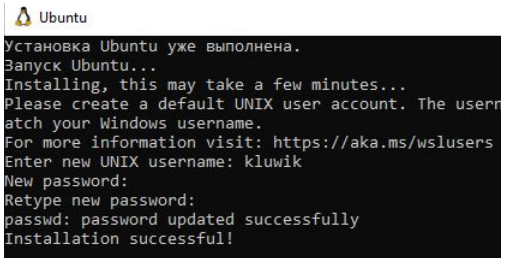


4. After installation finishes, **restart** the system.

5. After restart, WSL will autostart and set things up. It will take several minutes. Just wait.



6. When installation finishes you will be asked to set a username and password for your Linux system. After that you will be automatically logged in to the Linux system.



7. Profit! You now have Linux machine inside Windows. You can run it from Start menu: Start>Ubuntu.

Adapted from:
https://learn.microsoft.com/en-us/windows/wsl/install

Log in to cluster

To log in to cluster use the credentials that Lisa sent to you by email:

Open Terminal (for Linux/MacOS) or WSL (for Windows) and type:

```
ssh <username>@<server address>
```

- Username and password are case sensitive. The password will not be displayed when you type it.
- If you are asked about trusting the connection, type **`yes`** and press Enter.

Log in to cluster

Welcome to the computational cluster!

username

host name

home directory notation
(we will learn what is it later)

dollar sign denotes that you are logged in as a non-root (non-admin) user

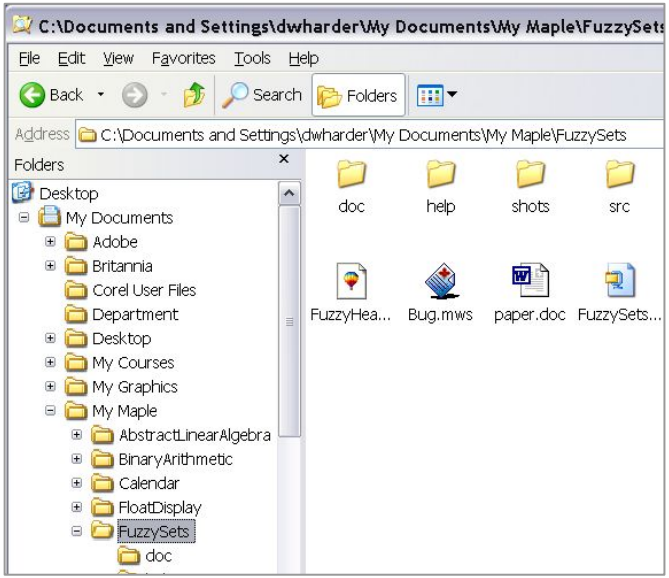
Navigation and directories

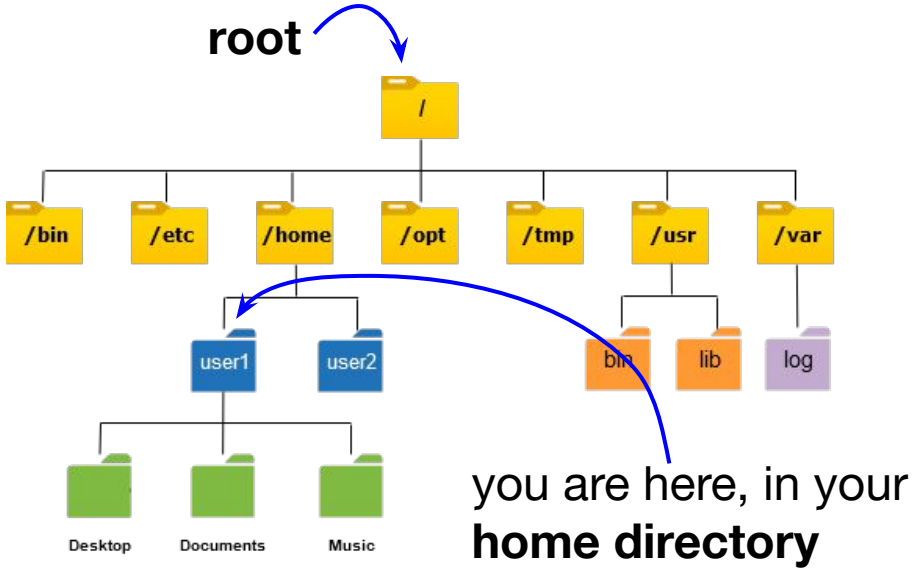
Let’s run your first command:

`pwd` – **p**rint **w**orking **d**irectory

```
[Marina.Pak@srv-khrameeva-01 ~]$ pwd
/home/Marina.Pak
```

Linux file system structure





you are here, in your **home directory**

(e.g. `/home/user/`)

- when you log in to server, you will appear in the **home directory**
- Windows: `\`, Unix: `/`

Directories: create directories

mkdir

Create a (sub)directory in your home directory:

`mkdir <directory_name>` – **m**a**k**e a **dir**ectory
a.k.a create a folder

example:
`mkdir seminar_10.11`

How to name files and directories:

- Don’t use spaces in the names. It is possible, but makes life harder.
- Forward slash **/** is prohibited (because it is directories separator).
- Give meaningful names, state dates. Avoid *untitled, untitled_2, final_final, new_folder*.

When you accidentally say “folder” instead of “directory” in a Linux group.

Directories: list contents

ls

Check the content of the directory:

```
[Marina.Pak@srv-khrameeva-01 ~]$ ls
seminar 10.11
```

`ls` – **l**i**s**t contents of the current directory

📌 It is possible to adjust the mode of operation of the command and specify input files/directories to commands using **flags** and **arguments**: `<command> -<flags> <argument>`

`ls -a` – display **a**ll files (including hidden files)

`ls -l` – display details (**l**ong format)

❓ How to display all files with details?

`ls -la`

Passing the directory name to `ls` will list its contents:

`ls seminar_10.11`

`ls /home/` – display files in the directory `/home/`

Manual and help

man, --help

📌 To learn what the command does and how to use it, run **`man <command>`** or **`<command> --help`**.
P.S. one-letter flags usually have prefix “-”, full-word flags have prefix “--”.

`man ls` or **`ls --help`**

❓ Being located in your current directory, create a directory with a subdirectory **`test_dir/test_subdir`**. Run manual of the command to learn how to do it.

```
mkdir -p command_line_seminar/test_dir
```

Autofill and navigation through the history of commands

📌 Start typing the directory and press *Tab* for autofill.

Use up and down arrows to navigate through the history of commands.

`history` – displays the history of commands

`!<command id>` – run the command by id

Ctrl + R – reverse searching through command history. Start typing the search word. To execute the command, press Enter. To continue search press Ctrl + R.

https://www.oreilly.com/library/view/learn-linux-shell/9781788995597/85d106f3-cc78-42d4-90d9-4a944db3c065.xhtml

📌 Ctrl+C – terminate a running command.
Ctrl+Z – suspend a running command. To resume, type **`fg`**.
Ctrl+D – closes the terminal window or ends terminal line input.

Directories: change directories

cd

Change directory:

`cd test_dir` – **c**hange **d**irectory to `test_dir`

`cd` – return to home directory

You can change to subdirectory directly without the need to change to parent directory first:

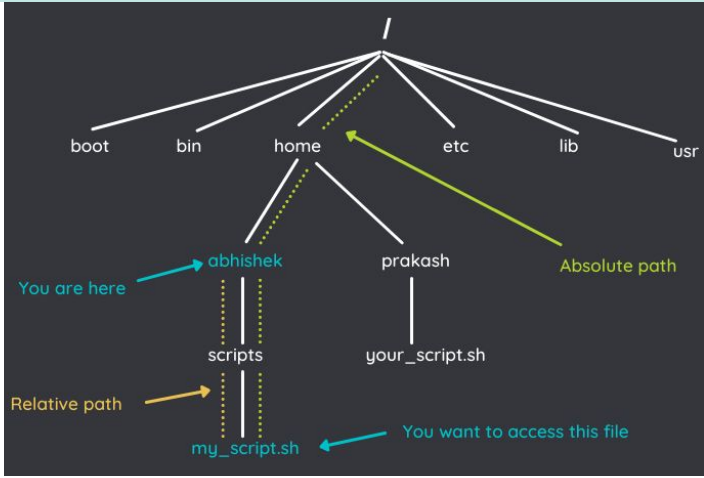
`cd test_dir/test_subdir` **relative** path

`pwd`

```
[Marina.Pak@srv-khrameeva-01 test_subdir]$ pwd
/home/Marina.Pak/test_dir/test_subdir
```

absolute or **full** path

Full and relative paths



Absolute (full) path – complete details needed to locate a file or directory, starting from the root element and ending with the other subdirectories.

Relative path – path related to the present directory.

Absolute path is always the same. With absolute path you can do manipulations with directory or file from any directory.

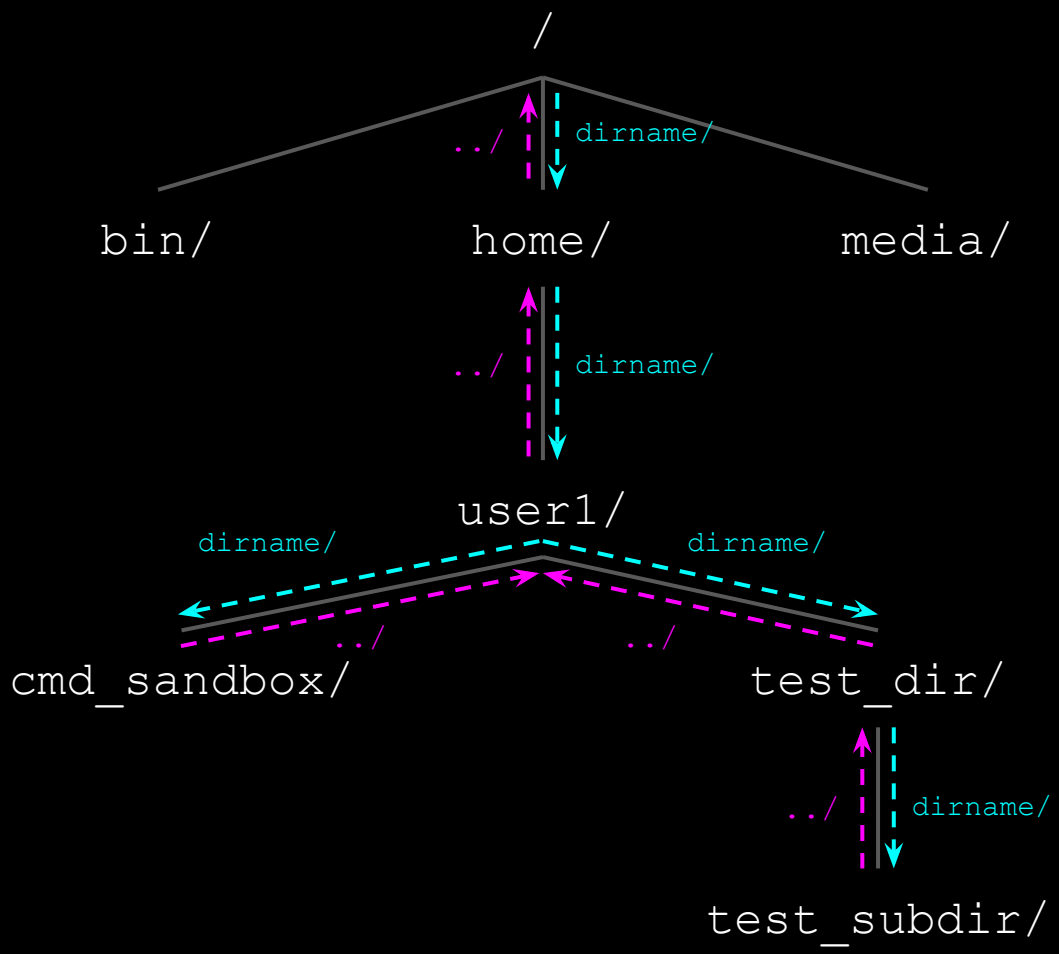
Relative path depends on the current directory. Relative paths never start with `/`.

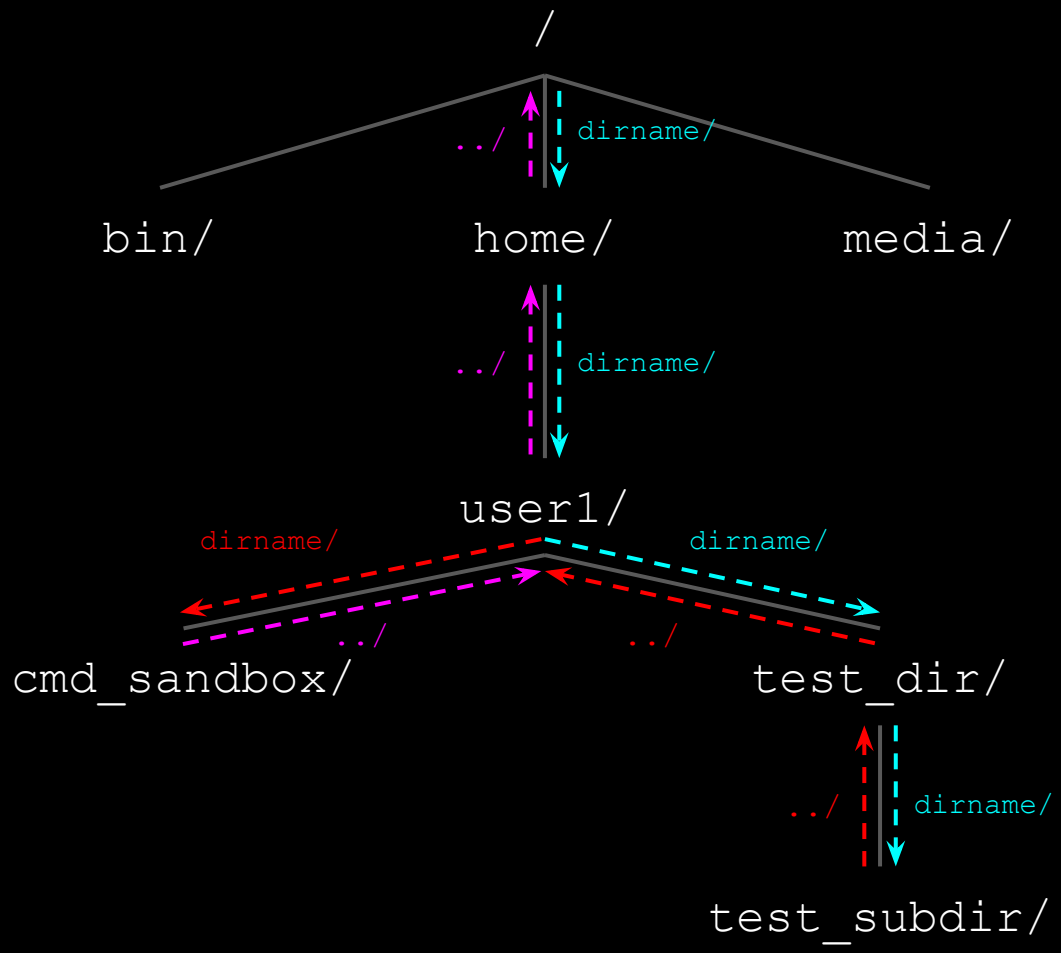
`/home/user/test_dir/test_subdir/` – absolute path

`test_dir/test_subdir` – path relative to **`/home/user/`**

`test_subdir/` – path relative to **`/home/user/test_dir/`**

❓ What is the path relative to **`/home/`**?


/
../
dirname/
bin/
home/
media/
../
dirname/
user1/
dirname/
dirname/
../
../
cmd_sandbox/
test_dir/
../
dirname/
test_subdir/



Path to `cmd_sandbox/` relative to `test_subdir/`
aka change to `cmd_sandbox/` from `test_subdir/`:
`cd ../../cmd_sandbox/`

`../` + `../` + `cmd_sandbox/` = `../../cmd_sandbox/`

References to directories in path

`/` – root directory

`~` – home directory

`.` – current directory

`..` – parent directory

❓ Make a relative path to:

a)	**`test_dir/`** from **`test_subdir/`**	**`..`**
b)	**`test_subdir/`** from home directory	**`test_dir/test_subdir/`**
c)	**`seminar_10.11/`** from **`test_subdir/`**	**`../../seminar_10.11`**
d)	home directory from **`test_subdir/`**	**`../../ or ~`**

Operations with files and directories

Create text files

touch, >, >>

1. **Create an empty file**

`touch <filename>` – creates a file if it doesn't exist or updates the last modified date if exists.

2. **Redirect terminal output (stdout) to file:**

`>` – write into file

```
ls > list.txt
```

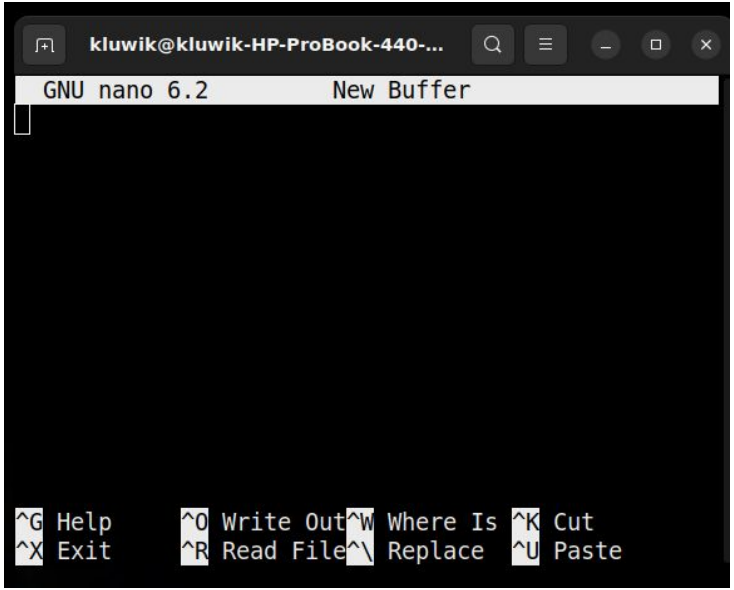
`>>` – append to the end of the file

```
echo "This line will be the last one!" >> list.txt
```

3. Text editors

- nano
- other

^ is Ctrl



Download files

wget

`wget <url>` – download file by url

```
wget https://rest.uniprot.org/uniprotkb/C7F6X3.fasta
```

Download the file, giving it a custom name:

```
wget https://files.rcsb.org/download/4NUH.pdb -O 4NUX_C7F6X3.pdb
```

Display contents of text files

cat, tac, less, more, head, tail

Full content, more suitable for small files:

`cat <file>` – display the content of the file

`tac <file>` – display the content of the file in reverse order

Full content, for long files:

`less <file>` – display the content of the file in a scrollable format

`more <file>` – display the contents of the file one screen at a time

Part of the content:

`head` – display the first ten lines of a file

- `head -<n>` – display the first n lines of a file

`tail` – display the last ten lines of a file

- `tail -<n>` – display last n lines
- `tail +<n>` – display lines starting from n-th

Move/copy/delete

mv, cp, rm, rmdir

`mv` – move = rename a file

`mv <old file name> <new file name>`

`mv <path to file> <new path to file>`

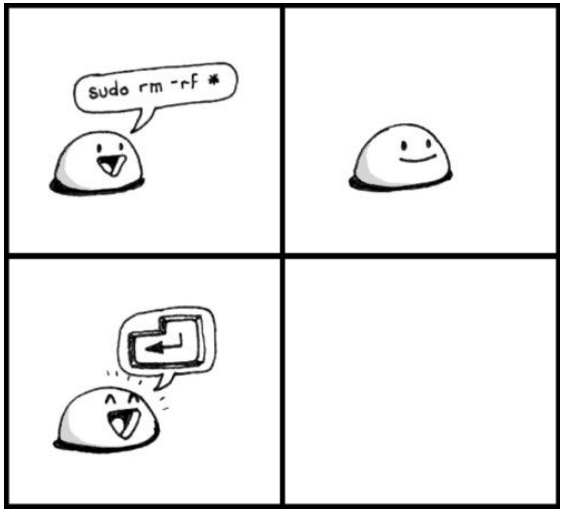
`cp <file> <directory>` – copy file into directory

`rm <file>` – delete file. Be very careful with this command.

`rmdir` or **`rm -r <directory>`** – delete directory

📌 To apply `cp` and `rm` to directories, use the flag `-r`.

https://askubuntu.com/questions/670648/what-does-rm-rf-do



https://neolurk.org/wiki/Rm_-rf

Command line. Part II

Bioinformatics course
Term 2, 2023
Skoltech

Chaining commands

You can run several commands sequentially in one line by using a **semicolon** `;`

```
cd ~/seminar_10.11; ls -la
```

```
touch file1.txt file2.txt; cp file2.txt file3.txt
```

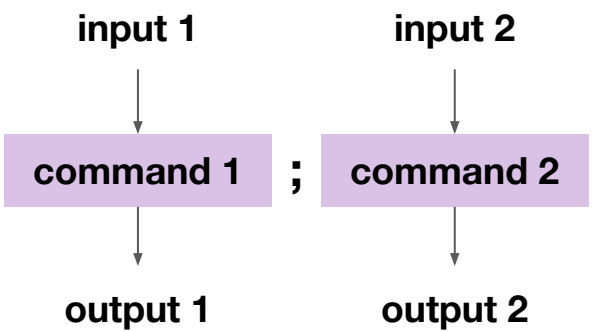
To redirect the output of the command to another command, pipeline `|` is used:

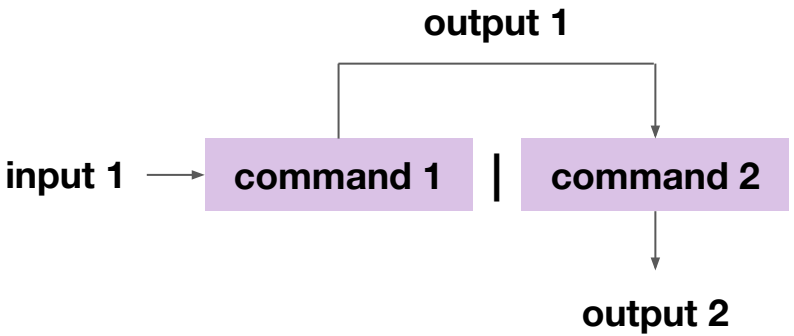
```
head 4NUX_C7F6X3.pdb | tail -1
```

```
echo 'I love command line <3' | wc -w
```

```
ls | wc -l
```

Chaining commands





Text processing

Text processing

The most popular tools for working with text in command line are `grep`, `sed`, `cut`, `tr`, and `awk`.

- `grep` – finds matching words and patterns in files
- `sed` – basic text transformations on an input stream (a file or input from a pipeline)
- `cut` – remove sections from each line of files
- `tr` – deletes, replaces, and converts characters
- `awk` – powerful scripting language with crazy syntax for text retrieval and processing

We will review the basic syntax and examples of frequently used commands.

There is no way to memorize or understand all parameters of these commands → constant googling.

For advanced text processing better use Python.

P.S. that's absolutely normal

Find matching word

grep

Let's play with the file **`/home/Shared/cu_binding.fasta`**

`grep` is an analog of Ctrl+F in GUI with extended functionality: **`grep <word> <file>`**

`grep P43561 cu_binding.fasta`

`grep '>sp' cu_binding.fasta` – use quotes when grepping special characters

`grep -c HUMAN cu_binding.fasta` – print number of matching lines

`grep -i cytochrome cu_binding.fasta` – ignore case

`grep '>sp' cu_binding.fasta | grep PIG` – use pipeline to grep multiple words

`grep '>tr' cu_binding.fasta| grep -v HUMAN` – inverse match

Find matching word

`grep` acts not only on text files but on any text inputs.

❓ a) List files that contain ‘file’ in the filename.

b) How many times the command cat was used?

```
ls | grep file
```

```
history | grep cat -c
```

Find matching pattern

grep

`grep` allows to search for **patterns** using **regular expressions (regex).**

Example of patterns:

- Digits — `[0-9]` or `\d` – range of digits from 0 to 9
- “The” in the beginning of the line — `^The` – ^ denotes the start of the line
- Capital letters — `[A-Z]` – range of letters from A to Z
- Words containing “bio” and “ics” — `\b\w*bio\w*ics\w*\b`
- Empty lines — `^$` – ^ and $ are start and end of line

`\b` – word boundary

`\w` – word character

`*` – zero or more occurrences

Patterns are specified by regular expressions using notations for classes and groups of characters, quantifiers, anchors, etc.

Find matching pattern

Let's play with the file **`/home/Shared/patterns.txt`**:

`grep [0-9] patterns.txt` – match digits

`grep [A-Z] patterns.txt` – match capital letters

`grep [A-Za-z] patterns.txt` – matchletters

`grep '\[' patterns.txt` – to match special characters, use an escape character `\`

`grep '^>' patterns.txt` – match lines starting from >. `^` denotes the start of the line

Find matching pattern

grep

https://cheatography.com/davechild/cheat-sheets/regular-expressions/

Anchors

^	Start of the line
$	End of the line
\b	Word boundary

Quantifiers

*	0 or more
+	1 or more
\+	At least once
{3}	Exactly 3
{3,}	3 or more
{3,5}	3, 4 or 5

Groups and Ranges

.	Any character except new line (\n)
[A-Z]	Upper case letter from A to Z
[0-7]	Digit from 0 to 7
[abc]	Range (a or b or c)
[^abc]	Not (a or b or c)
(a|b)	a or b

General tokens

\t	Tab
\n	New line
\s	White space
\S	Not white space
\	Escape character

Character classes

\d	Digit
\D	Non digit
\w	Word
\W	Non word
[:upper:]	Upper case letters
[:lower:]	Lower case letters
[:alpha:]	All letters
[:alnum:]	Digits and letters
[:digit:]	Digits
[:punct:]	Punctuation
[:blank:]	Space and tab
[:space:]	Blank characters
[:word:]	Digits, letters and underscore

Find matching word or pattern

grep

`grep` is cool:

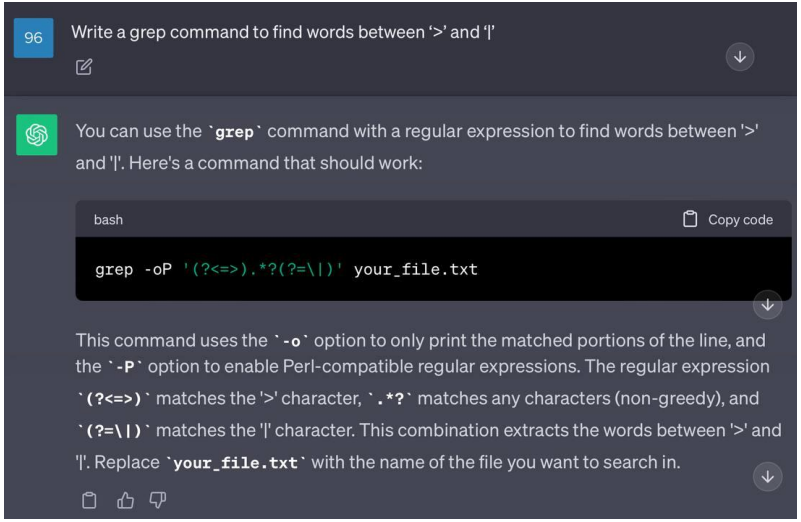
- finds words (analog of Ctrl+F)
- finds patterns using regex
- works with both text files and text inputs:
  ```
  grep <word or pattern> file.txt
  <command producing output to terminal> | grep <word or pattern>
  ```
- extracts pieces of text (use `-o` flag)
- in certain cases it is faster to use grep than other text processing tools:
  for example, to remove empty lines from a file you may simply use
  ```
  grep -v ^$ file_with_empty_lines.txt
  ```
  in Python you will have to firstly open the file, read it, remove empty lines, and save the file.

Find matching word or pattern

grep

How to compose a regular expression:

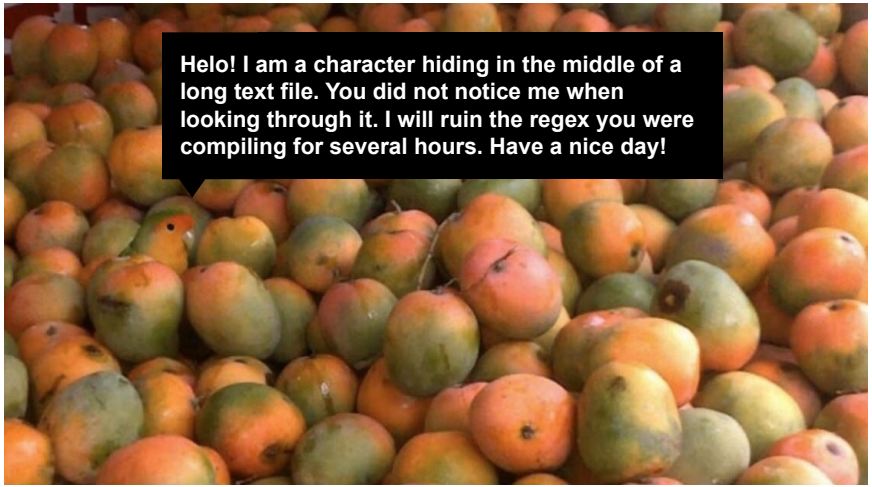
- Google
- regex101.com
- ChatGPT



Compiling a correct regex might be challenging:

- not obvious how to compile a regex (Google is your friend)
- pattern might be defined incorrectly

If it starts getting hard, consider using Python.



Delete / replace / convert specific characters

Basic text replace using `sed`:

`echo Hello, world! | sed 's/Hello/Hi/'` – replace *Hello* with *Hi*

`echo Hello, world! | sed 's/Hello, //'` – delete '*Hello, '* = replace with nothing

Another useful command is `tr`. It allows to delete (-d), replace (-s) and convert (-c) characters.

Convert lower case to upper:

```
echo small | tr '[a-z]' '[A-Z]'
```

Delete blank characters:

```
tr -d '[:space:]' < file.txt
```

Replace tab with commas:

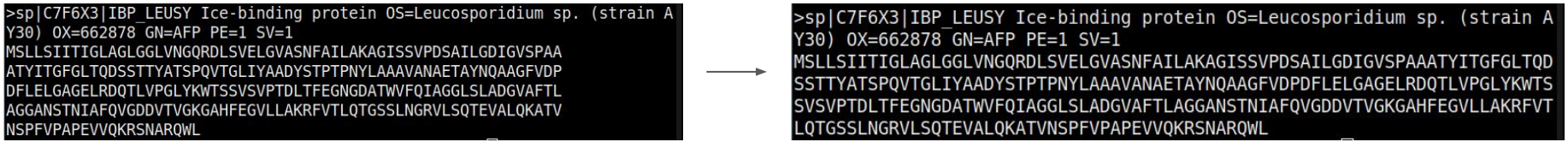
`cat file.tsv | tr '\t' ','` or `tr '\t' ',' < file.tsv`

https://linuxhint.com/50_sed_command_examples/

https://www.geeksforgeeks.org/tr-command-in-unix-linux-with-examples/

Practice

❓ Remove new lines from the sequence in the fasta file.
P.S. newline notation is **\n**




```
head -1 C7F6X3.fasta; tail -n +2 C7F6X3.fasta | tr -d '\n'
```

```
head -1 C7F6X3.fasta; grep -v '>' C7F6X3.fasta | tr -d '\n'
```

Scripts

Useful website about bash scripting: https://devhints.io/bash

Scripts

Bash script is a text file containing a sequence of commands that are executed by the bash program line by line.

Why scripts?

- no need to type commands one by one in terminal
- universal: a single script can be applied to any inputs

Bash script how-to:

1. The first line in the script should be `#!/bin/bash`. It tells the interpreter that it is a bash script.
2. Write your commands line by line.
3. Save the script with the extension `.sh`
4. Make the script executable: `chmod +x script.sh`
5. Run the script: `./script.sh`
   Note: if you run the script from the current directory, specify `./` before the name of the script.
   Script can also be launched by specifying the full path.

Scripts

Create a script in `nano`:

```
#!/bin/bash

head -1 C7F6X3.fasta > out.fasta
tail +2 C7F6X3.fasta | tr -d '\n' >> out.fasta
```

Save the script and run it:

```
chmod +x script.sh
```

```
./script.sh
```

Mind relative paths in scripts.

Note: first line and making the script executable might be omitted. In this case, the script is launched by the command `bash script.sh`

Scripts: input arguments

It is possible to supply input file(s) as **arguments** to a script.

Arguments passed to a script are referred in the script as `$1` (first argument), `$2` (second argument) and so on. `$` in bash scripting denotes a **variable**.

```
#!/bin/bash

head -1 $1 > $2
tail +2 $1 | tr -d '\n' >> $2
```

```
./script.sh C7F6X3.fasta out.fasta
```

Scripts: input arguments

❓ Write a script that takes the url of a fasta file as an argument, creates a directory **`downloads/`** with a subdirectory **`fasta/`** in home directory and downloads the file by url into the directory **`fasta/`**.
P.S. you may find useful the **`-P`** flag of **`wget`** command.

Make sure the script works when launched from any directory.
The current directory should not be changed after running the script.

```
#!/bin/bash

mkdir -p ~/downloads/fasta
wget $1 -P ~/downloads/fasta
```

📌 To leave comments in your script use **#** sign in the beginning of the comment line.
If the line of code is commented, it will not be executed.

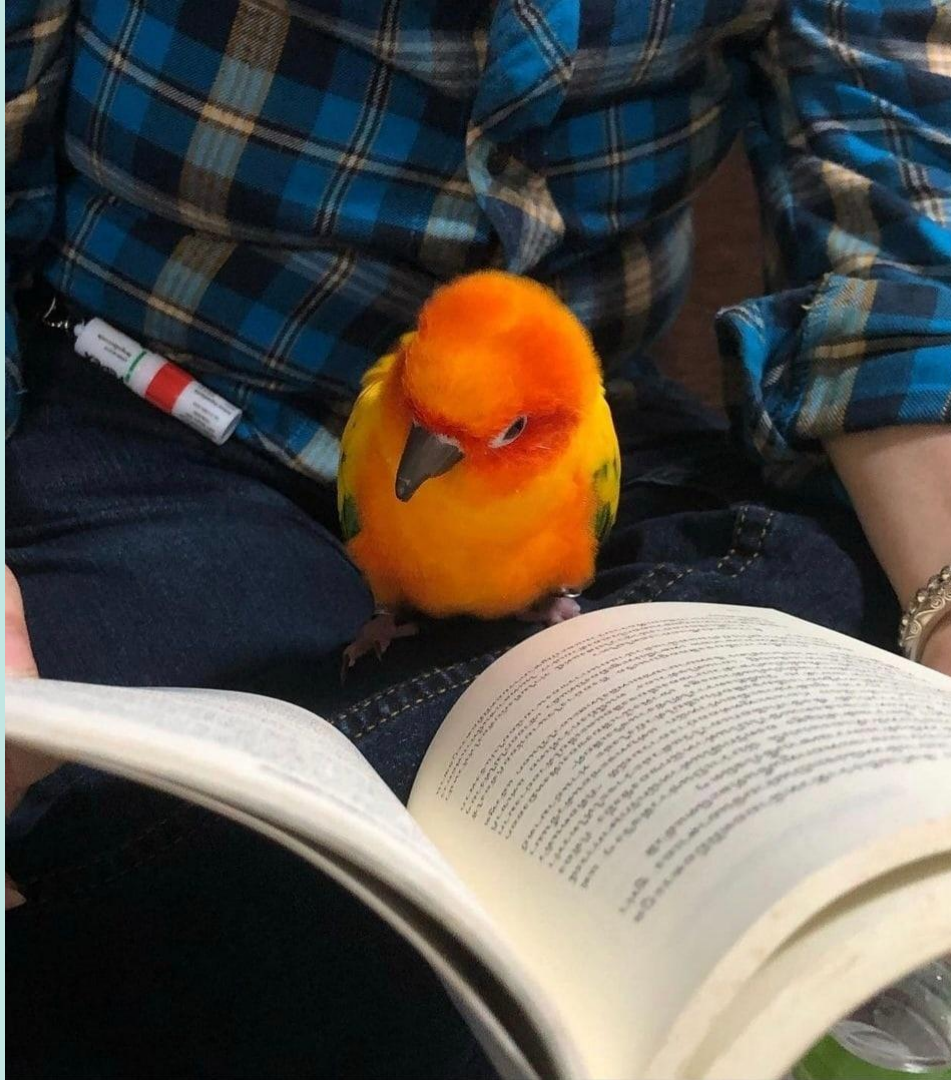
Scripts: variables

For convenience you may use variables in the script:

```
#!/bin/bash

DATA=~/downloads/fasta

mkdir -p $DATA
wget $1 -P $DATA
```

```
#!/bin/bash

DATA=~/downloads/fasta
uniprot_url='https://rest.uniprot.org/uniprotkb/'$1'.fasta'

mkdir -p $DATA
wget $uniprot_url -P $DATA
```

Self-study materials



Archives

File Extension	Bash Command to Compress	Bash Command to Extract
.zip	`zip -r archive.zip file`	`unzip archive.zip`
.tar	`tar -cvf archive.tar file`	`tar -xvf archive.tar`
.tar.gz	`tar -czvf archive.tar.gz file`	`tar -xzvf archive.tar.gz`
.tar.bz2	`tar -cjvf archive.tar.bz2 file`	`tar -xjvf archive.tar.bz2`
.gz	`gzip file`	`gunzip file.gz`

Transfer files

You can copy files and folders from and to the cluster via **`scp`**.

Type the **`scp`** command in cmd or terminal on **your local machine**, not on cluster!

Transfer file from cluster:

```
scp username@10.30.194.110:/home/user/file.txt /home/admin/Documents
```

your username on server	server address	**full** path to the transferred file	destination directory on your **local machine** (full or relative)

```
scp -r username@10.30.194.110:/home/user/folder /home/admin/Documents
```

Transfer file to cluster:

```
scp /home/admin/Documents/file.txt username@10.30.194.110:/home/user/
```

Print specific column

To print specific columns from file you may use either **`awk`** or **`cut`**.

Let's firstly download a table file to play with:

```
wget https://scop.mrc-lmb.cam.ac.uk/files/scop-cla-SARS-CoV-2.txt
```

Display the second and the third columns from comma-separated table:

```
awk -F' ' '{print $2,$3}' scop-cla-SARS-CoV-2.txt
```

P.S. `$` in bash denotes a variable.

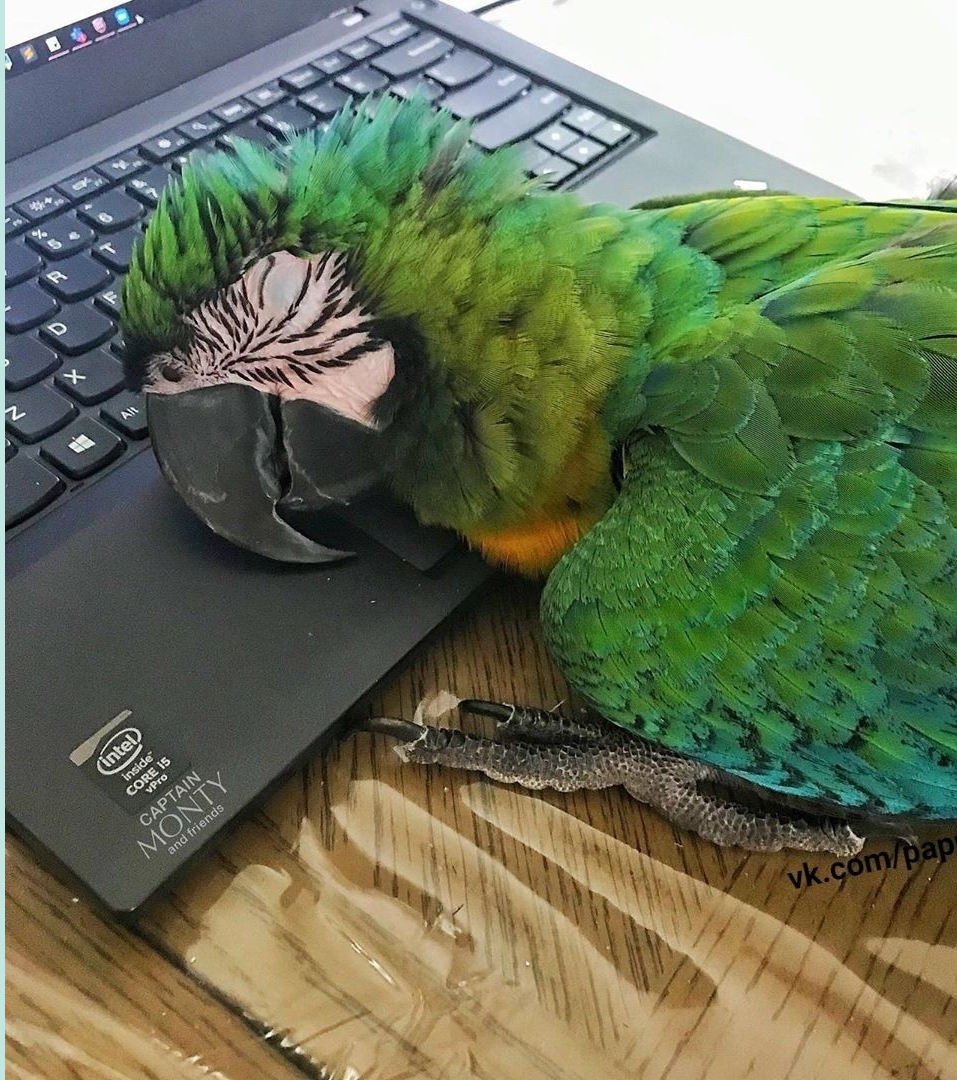
Delimiter (separator of the columns). In our case it is a single space character.

```
cut -d' ' -f2,3 scop-cla-SARS-CoV-2.txt
```

Print rows that contain "P0DTD1" in the fourth column:

```
awk '$4 == "P0DTD1"' scop-cla-SARS-CoV-2.txt
```

Useful tips and other commands



Running processes in `screen`

`screen` is an utility that creates a separate terminal session. Processes running in **`screen`** will continue to run when the **`screen`** window is detached even if you get disconnected.

`screen -S long_process` – create a screen named **`long_process`**

`screen -ls` – list all active screen sessions

`screen -r <screen id or name>` – resume screen

To detach the screen press **`Ctrl + a`** then **`d`** or type **`screen -d`** being in the screen session.
To kill the screen press **`Ctrl + a`** then **`k`** then **`y`**.

.bashrc

The **`.bashrc`** (located in the home directory) file is a configuration file for the Bash shell. The file consists of commands, functions, and scripts that are executed every time a Bash session starts.

The file allows customizing the shell environment with various functionalities, shortcuts, and visual tweaks.

Important! Before editing the **`.bashrc`** file make its backup (**`cp .bashrc .bashrc_backup_16.11`**). If something goes wrong you will be able to restore a working version of the file.

```
[Marina.Pak@srv-khrameeva-01 ~]$ ls -la
total 384
drwx------.  7 Marina.Pak Marina.Pak   4096 Nov 15 14:53 .
drwxr-xr-x. 62 root       root         4096 Nov 10 17:19 ..
-rw-------.  1 Marina.Pak Marina.Pak  12967 Nov 15 17:13 .bash_history
-rw-r--r--.  1 Marina.Pak Marina.Pak     18 Jul 27  2021 .bash_logout
-rw-r--r--.  1 Marina.Pak Marina.Pak    141 Jul 27  2021 .bash_profile
-rw-r--r--.  1 Marina.Pak Marina.Pak    413 Nov 24  2022 .bashrc
drwxrwxr-x.  2 Marina.Pak Marina.Pak   4096 Dec 13  2022 .ipynb_checkpoi
```

```
[Marina.Pak@srv-khrameeva-01 ~]$ cat .bashrc
# .bashrc

# Source global definitions
if [ -f /etc/bashrc ]; then
        . /etc/bashrc
fi

# User specific environment
if ! [[ "$PATH" =~ "$HOME/.local/bin:$HOME/bin:" ]]
then
    PATH="$HOME/.local/bin:$HOME/bin:$PATH"
fi
export PATH
export PATH=/opt/anaconda3/bin/:$PATH
# Uncomment the following line if you don't like systemctl's auto-paging feature:
# export SYSTEMD_PAGER=

# User specific aliases and functions
```

.bashrc: aliases

You can create a shortcut of the command using aliases:

```
alias alias_name='command_to_run'
```

```
alias cdsem='cd ~/seminar_10.11'
```

```
[Marina.Pak@srv-khrameeva-01 ~]$ alias cdsem='cd ~/seminar_10.11'
[Marina.Pak@srv-khrameeva-01 ~]$ cdsem
[Marina.Pak@srv-khrameeva-01 seminar_10.11]$
```

A useful alias to find commands in the history:

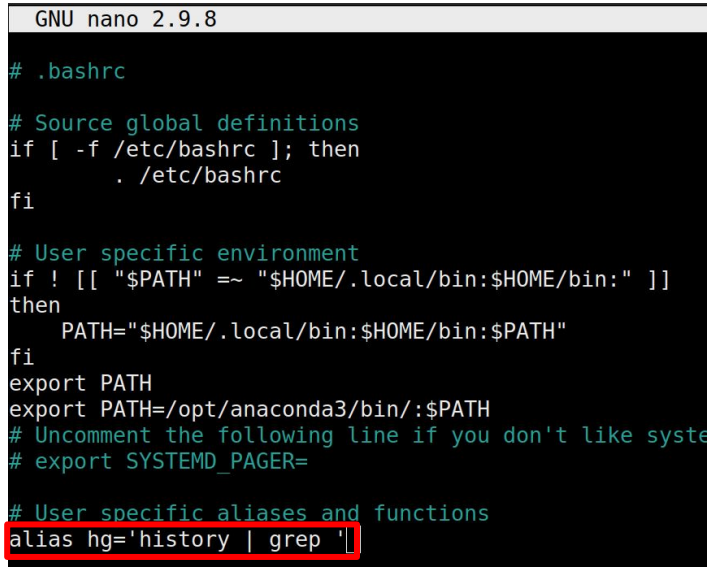
```
alias hg='history | grep '
```

```
[Marina.Pak@srv-khrameeva-01 ~]$ alias hg='history | grep '
[Marina.Pak@srv-khrameeva-01 ~]$ hg seminar
  216  mkdir seminar_10.11
  223  mkdir -p command_line_seminar/test_dir
  224  cd command_line_seminar/test_dir
  253  rm -rf command_line_seminar/ seminar_10.11/ C7F6X3.fasta 2brz.pdb
```

.bashrc: aliases

Aliases created in the terminal session will not be saved after you log out from the session.

To save aliases and use them anytime you need to write them into **`.bashrc`** file.



1. Open **`.bashrc`** in nano: **`nano .bashrc`**
2. Write your alias.
3. Save changes.
4. For changes to be applied in the current session type **`source .bashrc`**. Alternatively you could close the session and log in again.

.bashrc: PATH

The **PATH** is an environment variable that specifies paths to the directories with executable files.

It usually contains paths with Unix directories containing default executables and installed programs: **/bin**, **/usr/bin**, **/usr/local/bin.**

```
[Marina.Pak@srv-khrameeva-01 ~]$ echo $PATH
/opt/anaconda3/bin/:/home/Marina.Pak/.local/bin:
/home/Marina.Pak/bin:/usr/local/ncbi/sra-tools/b
in:/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/
sbin
```

When you call an executable by its name in the terminal, bash searches for it in the directories specified in the **PATH** and runs it when found it.

So, when we type bash commands like **ls**, **cd**, **cp** and other, bash looks for them in the directories listed in **PATH** and executes by a full path.

```
[Marina.Pak@srv-khrameeva-01 ~]$ which pwd
/usr/bin/pwd
[Marina.Pak@srv-khrameeva-01 ~]$ pwd
/home/Marina.Pak
[Marina.Pak@srv-khrameeva-01 ~]$ /usr/bin/pwd
/home/Marina.Pak
```

To see a full path to an executable, type **which <executable>**.

.bashrc: PATH

That is why we had to launch our seminar script by **`./script.sh`**, not by **`script.sh`**. In the first case, bash understands that the script is located in the current directory (**`./`** notation). In the second case it searches for **`script.sh`** in the directories specified in the **`PATH`** and does not find it: “command not found”.

```
[Marina.Pak@srv-khrameeva-01 seminar_10.11]$ ls
4NUX_C7F6X3.pdb  ice_protein.fasta  out.fasta
C7F6X3.fasta     newfasta.fasta     script.sh
[Marina.Pak@srv-khrameeva-01 seminar_10.11]$ ./script.sh
[Marina.Pak@srv-khrameeva-01 seminar_10.11]$ script.sh
-bash: script.sh: command not found
```

.bashrc: PATH

To add a folder to the **PATH** in the current terminal session, type:

```
export PATH=<absolute path to the folder>:$PATH
```

```
[Marina.Pak@srv-khrameeva-01 seminar_10.11]$ which script.sh
/usr/bin/which: no script.sh in (/opt/anaconda3/bin/:/home/Marina.Pak/.local/bin:/home/Marina.
Pak/bin:/usr/local/ncbi/sra-tools/bin:/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin)
[Marina.Pak@srv-khrameeva-01 seminar_10.11]$ export PATH=/home/Marina.Pak/seminar_10.11:$PATH
[Marina.Pak@srv-khrameeva-01 seminar_10.11]$ which script.sh
~/seminar_10.11/script.sh
```

To update **PATH** permanently add `export PATH=<absolute path to the folder>:$PATH` to **.bashrc** file.

```
 GNU nano 2.9.8                .bashrc                Modified

# User specific environment
if ! [[ "$PATH" =~ "$HOME/.local/bin:$HOME/bin:" ]]
then
    PATH="$HOME/.local/bin:$HOME/bin:$PATH"
fi
export PATH
export PATH=/opt/anaconda3/bin/:$PATH
export PATH=/home/Marina.Pak/seminar_10.11:$PATH
# Uncomment the following line if you don't like systemc
# export SYSTEMD_PAGER=

# User specific aliases and functions
alias hg='history | grep '

^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text
^X Exit      ^R Read File  ^\ Replace   ^U Uncut Text
```

Finding files

```
find <path where to search> -type f -name <what to search>
```

```
[Marina.Pak@srv-khrameeva-01 ~]$ find -name '*fasta'
./cu_binding.fasta
./out.fasta
./seminar_10.11/out.fasta
./seminar_10.11/ice_protein.fasta
./seminar_10.11/C7F6X3.fasta
./seminar_10.11/newfasta.fasta
```

```
[Marina.Pak@srv-khrameeva-01 ~]$ find -type d -name 'seminar_10.11'
./seminar_10.11
```

https://www.freecodecamp.org/news/how-to-search-for-files-from-the-linux-command-line/

Symbolic links

https://www.freecodecamp.org/news/linux-ln-how-to-create-a-symbolic-link-in-linux-example-bash-command/

Filename expansion (globbing)

Filename expansion is expanding filename patterns or templates containing special characters. Not exactly regex.

`ls *.txt` – list all files ending with `.txt`

`cp ???.?? ./dir` – copy files that have 3 letters in the name and 2 characters in the extension to folder `dir`

https://tldp.org/LDP/abs/html/globbingref.html
https://se.ifmo.ru/~ad/Documentation/Shells_by_Example/ch13lev1sec9.html

Slicing:

```
PDB='pdb4lzm.pdb'; echo ${PDB:3:4}
```

```
Output: 4lzm
```

Substitution:

```
PDB='pdb4lzm.pdb'; echo ${PDB//pdb/txt}
```

```
Output: pdb4lzm.txt
```

https://devhints.io/bash#parameter-expansions

Sorting, shuffling, removing duplicates

Sorting:

`sort inputfile.txt > outputfile.txt` – sort lines in a file and save a sorted file

`sort -u inputfile.txt` or `sort inputfile.txt | unique` – sort and remove duplicates

`sort -k 2 inputfile.txt` – sort by 2nd column (columns should be separated by space characters)

To shuffle lines or input range `shuff` command is used:

```
ls -lh | shuff
```